High Level Fault Simulation/Test Generation for RISC-V ALUs

Ayush Maral, Justin Hsu, Theo Rybnicek
Department of Electrical and Computer Engineering
University of California, Davis
Davis, CA, USA

Abstract - Arithmetic logic units (ALUs) are a critical component of computer architecture. Used to compute and process data, great lengths must be taken to ensure both functionality and correctness in design and manufacturing. This project presents a Verilog based testbench for design verification, and a software based verification tool designed to detect stuck-at-faults in ALUs by simulating logic behavior, manually injecting SSL faults, and generating test vectors. The testbench implements a verification technique that analyzes all ALU functions by applying edge cases and random vectors and comparing them with an expected output. The software based tools utilize five valued logic and use the PODEM algorithm to evaluate fault propagation. Through automated test vector generation and simulation, the tool is capable of identifying manufacturing defects and enables precise fault localization. The tools designed serves as a foundation for self-checking systems and facilitates further integration with other hardware testbenches and simulation for a more comprehensive, system based simulation.

Keywords: Arithmetic logic unit, stuck at faults, fault propagation, testbenches

I. Introduction

The arithmetic logic unit (ALU) serves as a fundamental functional unit in the core of any processor. It is responsible for executing the logical and integer operations as defined by the Instruction Set Architecture (ISA). As such, the ALU must be capable of handling a diverse array of operations-including arithmetic functions like addition, subtraction, and our extensions of multiplication, division and modulo, as well as logical operations such as AND, OR, XOR, NAND, NOR, XNOR, and bit shifting operations like shift left and right. As programs grow in complexity, the probability of errors in design or manufacturing increase, resulting in the need for rigorous verification and validation of the ALU to ensure system stability and functionality. By employing a self checking testbench in the pre-silicon phase, designers can catch functional errors, preventing costly iterations and time investment.

Due to the geometries of transistors and inherent process variations in manufacturing, perfect devices can not be guaranteed. This could lead to incorrectly built transistors, shorted wires, open circuits, or other manufacturing defects that prevent the proper operation of the device.

Single stuck-at faults (SSF) are one of the most common errors that occur in the physical implementation of digital circuits. An errant wire shorted to ground or VDD could not only sever a connection between the output and its downstream input, but also contaminate the downstream results due to an input being forced to a certain value. As a result, test vectors must be applied to detect whether these stuck at faults occur by propagating the currently tested signal through to the output. Manually or randomly generating test vectors is not feasible due to the sheer number of available vectors to choose from. As such, automatic test generation must be employed to carefully choose the correct vectors relevant to propagating faults.

Automatic test generation can use a variety of different algorithms, but we chose to implement Path Oriented Decision Making

(PODEM) due to its effectiveness in generating collapsed lists of test vectors.

A. PODEM

1. **Fault Activation:** PODEM begins by identifying a target fault-a stuck at fault for our case. PODEM then assigns values to the inputs to propagate to the output.
2. **Fault Propagation:** Inputs are chosen in such a way that the fault is able to propagate all the way to the output of the circuit.
3. **Backtracking:** PODEM uses a decision driven backtracking process in which propagation is justified through careful selection of primary input values. If a certain decision for primary input does not result in the propagation of a fault, PODEM backtraces and tries an alternative value of the primary input, repeating this process until the fault has been fully propagated to the output and all the primary inputs have been assigned values to become an input vector. This sequence repeats until a test vector is found or all possibilities are exhausted.

This project applies PODEM to a 32 bit ALU, which not only functionally verifies the ALU as an independent unit, but also serves as the one of the first tests in ensuring functionality of a processor. This is not only of utmost importance due to the critical nature of the ALU in processor operation, but also due to the sheer complexity of modern processors. With modern processors implementing complex systems such as simultaneous multi-threading and pushing a dozen cores, the ALU sits at the heart of operation-executing basic processing logic to fuel programs. Due to the versatility and importance of the ALU, we chose to develop frameworks for verification and testing a system with such a vital role in modern day computation.

II. Methodology and Implementation

A. Module Design and Testbench Logic

The primary objective was to develop a robust, 32 bit RISC-V compliant ALU using Verilog. Additional ISA extensions were implemented as well-adding functions like multiply, divide and modulo. The ALU takes a 32 bit input, and decodes the RISC-V machine code. Through the decode process, a function is selected through the opcode section, and a target register is selected and source registers are chosen. Next the opcode is used to select what operation to perform using a series of case statements. After a function is chosen, the function is evaluated using the source registers and written to the target register. Functions are defined behaviorally, with the exception of the logical functions where logic gates were instantiated instead. Exceptions such as overflow, zero output, and divide by 0 are triggered due to logic testing the output of the evaluated function and output as flags.

To verify functionality of the RTL design, a self-checking testbench was developed. This tool automatically compares the output of the ALU under test with software calculated outputs (the golden model) of the same test, and outputs the results to the console. The display output shows the two inputs, the output of the ALU, the output of the software based calculation, which flags were raised (if any), and whether the two outputs match and whether the test passes or fails. At the end, the testbench outputs how many tests passed, and how many

were failed. For each individual function of the ALU, random inputs were applied to simulate the average operation of an ALU. Next, edge cases were applied. For example, dividing a number by 0, or multiplying a number so the result is an overflow error. Edge cases were specifically chosen such that they represent the worst case scenario and might break the system due to causing instability from unexpected inputs. This system for verification continues for each function supported by the ALU, while the testbench compares the output with the golden model in real time to determine whether any errors in design have occurred. If a mismatch is detected, an error is output detailing what output of the ALU was, and what the golden model output, along with any relevant flags. This allows for precise localization of logic faults within the RTL itself, and helps designers pinpoint exactly what went wrong.

B. Structural ALU Implementation in Python

To complement the Verilog RTL implementation and enable systematic fault analysis, a structural model of the 32-bit RISC-V ALU was implemented in Python. Unlike the Verilog design, which describes the ALU behavior using high-level case statements and RTL abstractions, the Python implementation models the circuit structurally by representing individual wires and logic units explicitly in software. This approach preserves the connectivity of the circuit and enables precise injection of faults at specific internal nodes.

The structural simulator is built around two primary abstractions: Wire objects and Gate objects. A Wire represents a named signal in the circuit and carries an integer value representing the binary state of that signal. Wires can be either single-bit or multi-bit signals, allowing the model to represent both internal control signals and the full 32-bit operand buses used by the ALU. Each wire also maintains internal state information that allows individual bits to be forced to specific values when faults are injected.

A Gate represents a functional block that operates on one or more input wires and produces an output wire. The Python model implements gate classes corresponding to each of the ALU operations. Logical gates such as AND, OR, XOR, and NOT evaluate their outputs using Python's bitwise operators across all 32 bits simultaneously. Arithmetic operations are implemented through specialized classes such as FullAdder32, Subtractor32, Multiplier32, Divider32, and Remainder32. The FullAdder32 class performs addition using a ripple-carry approach, computing the sum bit-by-bit from least significant to most significant bit. Subtraction is implemented using two's complement arithmetic by adding the bitwise complement of the second operand plus one.

Additional functional units model the shift and comparison operations supported by the ALU. ShiftLeft32 and ShiftRight32 extract a five-bit shift amount from the lower bits of the second operand and perform logical shifts accordingly. The Comparator32 unit implements the set-less-than operation by comparing the unsigned values of the operands and producing a single-bit result.

All functional unit outputs are connected to a multiplexer implemented by the Mux32 class. This multiplexer selects one of the operation results based on the four-bit ALU_Sel input signal and routes the chosen value to the final output wire ALU_Out. Each functional unit result is therefore always computed, but only the result selected by the multiplexer becomes externally observable.

The complete ALU is encapsulated within a StructuralALU class that instantiates every wire and gate and connects them into a netlist representing the full circuit. To evaluate the ALU, input values are assigned to the primary input wires A, B, and ALU_Sel, after which each gate is evaluated in sequence. The resulting value can then be read from the output wire ALU_Out.

To verify correctness of the structural model, a golden reference implementation of the ALU was written in Python using standard arithmetic and logical operators. For each test vector, the structural model output is compared with the golden reference result. If the values differ, the simulator reports a mismatch along with the corresponding input vector and operation. Using this verification approach, a suite of directed test vectors and random test vectors was applied to the structural model. The combined test suite validated all supported operations, including addition, subtraction, logical operations, shifts, comparisons, multiplication, division, and modulo, and confirmed that the structural simulator produces identical results to the golden reference model.

C. Fault Injection in the Structural ALU Implementation

A key objective of the Python structural model is to enable systematic fault injection for evaluating the testability of the ALU. The simulator implements a single stuck-at-fault (SSF) model, which represents manufacturing defects where a wire is permanently forced to logic 0 or logic 1.

Fault injection is implemented directly within the Wire class. Each wire maintains a dictionary of faults specifying which bit positions are forced to a stuck value. When the wire value is read during gate evaluation, the simulator applies the fault mask to override the computed value. For a stuck-at-0 fault, the affected bit is forced to zero; for a stuck-at-1 fault, the bit is forced to one. Because the fault correction occurs when the wire value is accessed, the fault automatically propagates through the remainder of the circuit without requiring any modifications to the gate logic itself.

Two forms of fault injection are supported. The first is whole-wire faults, where every bit of a signal is forced to a fixed value, modeling conditions such as a bus line shorted to ground or supply voltage. The second is bit-level faults, where a single bit within a multi-bit wire is forced to a value. Bit-level injection more accurately models localized defects occurring in a specific wire of a bus.

To evaluate fault coverage, a comprehensive list of possible faults is generated by iterating through every wire in the ALU and creating two faults for each bit position: stuck-at-0 and stuck-at-1. For the 32-bit input and output buses, the operation result wires, and the selector signals present in the design, this process produces hundreds of possible faults distributed across the entire circuit.

During fault simulation, each fault is injected individually into the ALU model. The full test vector suite is then executed, and the faulty output is compared against the golden reference output. If any test vector produces a discrepancy between the two outputs, the fault is considered detected. By repeating this process for every possible stuck-at fault, the simulator determines the overall fault coverage of the test set.

This structural simulation environment provides a flexible platform for analyzing how faults propagate through the ALU and evaluating the effectiveness of test vectors in detecting manufacturing defects. Because each internal signal is explicitly modeled and accessible, the simulator can precisely identify where faults occur and how they influence the final output.

This capability makes the Python model an effective complement to the Verilog testbench, enabling deeper analysis of fault behavior and supporting the development of automated test generation techniques such as the PODEM algorithm.

D. PODEM Test Generator

The PODEM test generator was implemented in Python as a companion to the structural ALU simulator, designed to automatically derive input vectors capable of detecting each fault in the key fault list sourced from the structural model. For a given target fault and a wire bit stuck at logic 0 or logic 1 the algorithm proceeds through three repeating phases. First, an objective function determines which primary input bit to drive and to what value, prioritizing the assignment of ALU_Sel to route the faulty internal wire through the output multiplexer, then backtracing through the relevant gate logic to identify a controlling primary input. Second, the derived objective is forward implied: the selected input bit is assigned, and the partial input state is evaluated against the faulty and fault free simulation models. If the outputs diverge, a test vector has been found. Third, if a contradiction arises meaning a prior assignment conflicts with the new objective the algorithm backtracks by reversing the most recent decision and attempting its complement, repeating until detection is achieved or all possibilities are exhausted. The backtrace rules are implemented analytically for each gate type, following standard D-calculus principles: AND gates require both inputs at the non-controlling value of logic 1 for target 1, while a single controlling logic 0 suffices for target 0; OR and XOR gates follow symmetric reasoning; and multi-operand units such as ADD and SUB suppress carry interactions by zeroing the lower bits of one operand before asserting the target bit. Notably, the REM operation required special handling for the most significant bit, where the pattern $A = 2^b$ and $B = 2^b + 1$ was used to guarantee $A < B$, ensuring $A \% B = A$ and making bit b of the result directly observable.

III. Results and Discussion

A. Testbench Results

The integration of the self-checking testbench significantly improved efficiency in verifying functionality compared to manual analysis. Through real time comparison to the golden model, the testbench allows for much faster verification and higher confidence in the functionality of the unit. In total 56 tests were applied to the ALU, with each of the 12 functions tested and evaluated 4 times using random inputs, and several functions additionally tested using edge cases such as forcing a divide by 0 or multiplying something that would result in an overflow. In total, all 56 applied tests passed, with a total of 0.053 seconds spent on running the testbench. This highlights the speed of the self-checking testbench, and the confidence it instills in a design.

```
--- DEFAULT (opcode 4'b1111) ---
PASS | Test 49 | DEF  | A=3735928559 B=3405691582 | Out=0 | Expected=0 | Z:1 O:0 D:0

--- FLAG EDGE CASES ---
PASS | Test 50 | ADD_O | A=2147483647 B=1 | Out=2147483648 | Expected=2147483648 | Z:0 O:1 D:0
PASS | Test 51 | SUB_O | A=2147483648 B=1 | Out=2147483647 | Expected=2147483647 | Z:0 O:1 D:0
PASS | Test 52 | MUL_O | A=2147483647 B=2 | Out=4294967294 | Expected=4294967294 | Z:0 O:1 D:0
PASS | Test 53 | DIV_Z | A=100 B=0 | Out=4294967295 | Expected=4294967295 | Z:0 O:0 D:1
PASS | Test 54 | DIV_O | A=2147483648 B=4294967295 | Out=2147483648 | Expected=2147483648 | Z:0 O:1 D:0
PASS | Test 55 | REM_Z | A=100 B=0 | Out=100 | Expected=100 | Z:0 O:0 D:1
PASS | Test 56 | REM_O | A=2147483648 B=4294967295 | Out=0 | Expected=0 | Z:1 O:1 D:0

==================================================
 Results: 56 PASSED,  0 FAILED  (Total: 56)
==================================================
ALU.v:459: $finish called at 570 (1s)
[Done] exit with code=0 in 0.053 seconds
```

Figure 1. Output of self-checking testbench

B. Python Structural Model and Fault Simulation Results

To further evaluate the ALU design, a structural model of the ALU was implemented in Python and used to perform fault injection experiments. After verifying functional correctness against a golden reference model, the structural simulator was tested using a combination of directed and randomly generated input vectors. In total, 251 test vectors were applied to the Python ALU model, consisting of both targeted edge cases and random operand combinations. All test vectors produced outputs identical to the golden reference implementation, confirming that the structural model accurately replicates the intended ALU functionality. The structural simulator was then used to evaluate the impact of single stuck-at faults (SSF) within the ALU. Faults were injected at the bit level on each wire in the design, producing a total of 906 possible stuck-at faults across the ALU. Each fault was evaluated individually by executing the full test vector suite and comparing the faulty output to the golden reference output. A fault was considered detected if any test vector produced a mismatch. Using the combined test set, the simulator achieved 100% fault coverage, meaning every injected fault was detected by at least one input vector. The results also illustrate how certain faults are more difficult to detect than others due to structural characteristics of the ALU. In particular, faults occurring within individual functional units can be masked by the output multiplexer when that operation is not selected. This highlights the importance of including operation-specific test vectors to ensure complete fault coverage.

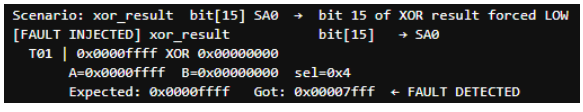

Figure 2. Output of fault injection system

C. PODEM Test Generator Results

Applied to the 94 faults in the critical fault list covering primary inputs, control signals, the primary output, and all internal result buses, the PODEM generator achieved 100% fault coverage, producing a verified test vector for every target fault with each fault resolved in a single iteration.

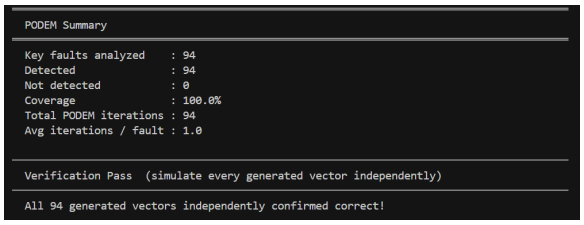

Figure 3. Output of PODEM test generator

IV. Conclusions and Future Work

This project demonstrated a comprehensive framework for verifying and analyzing a 32-bit RISC-V ALU using both hardware and software based approaches. A Verilog implementation of the ALU was first validated using a self-checking testbench that compared outputs against a software based golden model. Through directed edge cases and randomized testing, the testbench confirmed correct operation across all implemented ALU functions while providing fast and automated verification.

To extend the verification process beyond functional correctness, a structural ALU model was developed in Python. This model represents the internal wires and functional units explicitly, enabling controlled fault injection and detailed observation of signal propagation. Using this environment, single stuck-at faults were injected across the ALU and evaluated using a combined set of directed and random test vectors. The simulator successfully detected all injected faults, achieving full fault coverage and

demonstrating the effectiveness of the test vector set.

In addition, a Python implementation of the Path Oriented Decision Making (PODEM) algorithm was developed to automatically generate test vectors capable of detecting difficult faults. By analytically backtracing through gate logic and iteratively assigning input values, the PODEM generator was able to produce valid detecting vectors for every fault in the critical fault list. Together, the Verilog testbench, structural simulator, and automated test generator form a layered verification methodology capable of validating functionality, identifying manufacturing defects, and producing targeted test vectors.

Future work could expand this framework in several directions. One potential extension is the integration of additional fault models such as bridging faults, delay faults, or transient faults, which more closely represent real manufacturing defects in modern CMOS technologies. Another improvement would be scaling the structural simulation framework to support larger components of a processor pipeline, allowing similar fault analysis to be performed on register files, control logic, or memory interfaces. Performance optimizations could also be implemented to accelerate large scale fault simulation, enabling the evaluation of significantly larger test vector pools. Finally, integrating the Python based PODEM generator directly with the Verilog simulation environment would allow automatically generated test vectors to be applied to the RTL design, creating a unified verification flow that bridges high level test generation with hardware level validation.

V. References

[1] H. Al-Asaad, Lifetime Validation of Digital Systems via Fault Modeling and Test Generation, University of Michigan, 1998

[2] S. C, M. E, M. K. D, J. K and J. L. J, "Efficient Design and Functional Testing of a 32-Bit RISC Processor in Verilog," 2025 3rd International Conference on Smart Systems for applications in Electrical Sciences (ICSSES), Tumakuru, India, 2025, pp. 1-6, doi: 10.1109/ICSSES64899.2025.11009267. https://ieeexplore.ieee.org/document/11009267

[3] O. Cekan, R. Panek and Z. Kotasek, "Input and Output Generation for the Verification of ALU: A Use Case," 2018 IEEE East-West Design & Test Symposium, Kazan, Russia, 2018, pp. 1-6, doi: 10.1109/EWDTS.2018.8524641. https://ieeexplore.ieee.org/document/8524641

[4] R. Verma, H. C S and L. Shrinivasan, "Functional Verification of Arithmetic Logic Unit and Instruction Fetch Unit of a 32-bit RV321M ISA based RISC-V Processor," 2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT), Bangalore, India, 2024, pp. 1-5, doi: 10.1109/CONECCT62155.2024.10677337. https://ieeexplore.ieee.org/document/10677337

[5] E. Cerny, E. M. Aboulhamid, G. Bois and J. Cloutier, "Built-in self-test of a CMOS ALU," in IEEE Design & Test of Computers, vol. 5, no. 4, pp. 38-48, Aug. 1988, doi: 10.1109/54.7968. https://ieeexplore.ieee.org/document/7968

[6] D. Gizopoulos, A. Pachalis, Y. Zorian and M. Psarakis, "An effective BIST scheme for arithmetic logic units," Proceedings International Test Conference 1997, Washington, DC, USA, 1997, pp. 868-877, doi: 10.1109/TEST.1997.639701. https://ieeexplore.ieee.org/document/639701